

THE STRUCTURE OF MATHEMATICAL REALITY

Phase IV — Chapter 15

Algorithmic Complexity and the Search for Generative Equations

Lecture

Self-Study Curriculum

This chapter formalizes the idea you arrived at independently: that finding the generative equation behind data is a form of compression, and that this form of compression is qualitatively different from statistical encoding. You reasoned your way to this before you had the vocabulary, and you went one step further, asking whether the search space of equations itself could be made variable. Both the intuition and the follow-up question have exact homes in the landscape, and this chapter installs them. It also delivers the two hardest limitative results in the curriculum, the unsolvability of the halting problem and the uncomputability of the quantity we are about to define, and it does so not as announcements but as consequences of a single move you have already watched succeed three times.

Aside

A necessary disambiguation, two things both called “complexity”: The word *complexity* names two entirely different quantities in this curriculum, and conflating them is the single most common error a newcomer makes here.

- **Kolmogorov (descriptive/algorithmic) complexity**, this chapter, measures the *length of the shortest program* that produces an object. It asks: how short can the description be? It is about *size*, and it is *uncomputable*.
- **Time (computational) complexity**, the subject of Chapter 5’s hardness side and of P vs. NP, measures *how many steps* a computation takes. It asks: how long does it take to run? It is about *cost*, and for a given algorithm it is perfectly computable.

A string can have tiny Kolmogorov complexity yet be expensive to compute (the first billion digits of π : a two-line program, but real work to run), and the two notions can even pull in opposite directions. When this chapter says “complexity,” it always means the *descriptive* kind. Keep the two on separate shelves.

The machine underneath: what “program” means

The definition at the center of this chapter rests on the word *program*, so pin that word down first, because a definition built on a vague word is worthless. In 1936, five years after Gödel and as the cleaner sequel to him, Alan Turing gave the first fully mechanical definition of computation: a machine with a finite table of rules, reading and writing symbols on an unbounded tape, moving left or right one cell at a time. The hardware is beside the point; the tape and the head are a cartoon. The point is the thesis that came with it, the **Church-Turing thesis**: every step-by-step procedure, anything that could in principle be carried out by rote with paper, pencil, and patience, can be carried out by such a machine. This is not a theorem, because “could be carried out by rote” is not a formal notion; it is a claim about the reach of an idea, and its evidence is overwhelming. Every general-purpose model of computation anyone has proposed since (lambda calculus, register machines, cellular automata, every programming language you have used, the processor in your laptop) computes exactly the same class of functions. Nothing has ever escaped it. “Program” is therefore not a technology-dependent notion, not something that will mean more in ten years; it is a structural one, fixed for good, and that is the only reason the definition below is well-posed rather than a statement about whatever hardware happens to be lying around.

One more fact makes the definition canonical rather than arbitrary. There exist **universal** machines: single, fixed machines that simulate any other machine when handed that machine’s rule table as input. Your laptop is one. It is a fixed object, yet it runs arbitrary software, because

software is just a description of a machine and the laptop is a universal simulator of described machines. Fix one universal machine U and you can define the length of the shortest program for anything relative to U . Swap U for a different universal machine and every length shifts by at most a fixed constant, the length of a one-time translator that teaches the new machine to imitate the old one. That constant does not depend on the object you are describing, so for large objects it washes out, which is why the choice of U does not matter for anything structural. This is the *invariance theorem*, and it is what lets us speak of *the* complexity of a string rather than its complexity on some particular machine.

The halting problem, the diagonal returns

Turing proved, in the same 1936 paper, that there are precise, well-posed questions about programs that no program can answer. The flagship is the **halting problem**: given a program and its input, does it eventually halt, or run forever? This is not a vague question. For any specific program you can often answer it by hand. The claim is that no *single* program can answer it for *all* programs, and the proof is a machine wearing Cantor's clothes.

Suppose, for contradiction, that you had a decider $\text{HALTS}(p, x)$ that always answers correctly, returning “halts” or “runs forever” for any program p on any input x , always terminating with the right answer. Use it to build a saboteur D . On input p (a program's own text), D asks HALTS what p does when fed *its own text*, and then does the opposite: if HALTS says “ p halts on p ,” then D loops forever; if HALTS says “ p runs forever on p ,” then D halts immediately. D is a perfectly ordinary program; if HALTS exists, D can be built from it. Now run D on its own text. Ask what happens. If D halts on D , then by D 's construction HALTS must have said “runs forever,” but HALTS is never wrong, contradiction. If D runs forever on D , then HALTS must have said “halts,” again a wrong answer from an infallible decider, contradiction. Every branch is a contradiction, so the one assumption we made must be false: HALTS cannot exist. There is no general procedure that decides halting.

Look at the shape of that argument, because the shape is the point of the whole chapter. It is Cantor's diagonal from Chapter 2 in a new costume. There, you listed every real number and built one that differs from the n -th on the list at the n -th digit, so it is on no list; here, D is built to differ from every possible decider's verdict at one strategically chosen input, its own text, so no decider is correct about it. And it is the same self-reference that powers Gödel's incompleteness theorem in Chapter 4, where a formula is engineered to say, in effect, “I am not provable.” The three results are not merely similar; they are joined. If every true statement of the form “program p never halts” were provable in some fixed formal system, you could decide halting mechanically by searching all proofs in parallel until one settled the question, so a complete-and-correct proof system would *solve* halting. Turing says nothing solves halting. Therefore no such complete system exists: the unsolvability of halting *forces* incompleteness. Cantor in 1874, Gödel in 1931, Turing in 1936: one diagonal, three escalating conclusions, from “some infinities are bigger” to “some truths are unprovable” to “some questions are undecidable.” This triangle is connective tissue in exactly the sense this curriculum is built to expose, and it is the wall that the next definition is about to run straight into.

Kolmogorov complexity

Here is the quantity the whole chapter is organized around. The **Kolmogorov complexity** of a string x is the length of the shortest program, on a fixed universal machine U , that outputs x and halts.

$$K(x) = \min\{ |p| : U(p) = x \}$$

Read it slowly. Among all programs that print x and stop, take the shortest; its length in bits is $K(x)$. This is the ultimate measure of the information content of an object, and it is ultimate in a strong sense: it does not care where x came from, what process produced it, or what you believe about it. It asks only how briefly x can be *regenerated*.

Two poles make the idea concrete. If x is a highly structured string, say a million repetitions of “01,” then $K(x)$ is tiny: the program “print 01 a million times” is a few dozen symbols regardless of how long x is, because the structure can be exploited to write a short generator. If x is a genuinely patternless string, $K(x)$ is essentially $|x|$ itself, because the shortest program is basically “print this exact string,” which has to contain the string and so cannot be much shorter than it. Structure is exactly the gap between $|x|$ and $K(x)$: the more a string can be compressed, the more structure it has, and the size of the shortest program is the precise, machine-independent measure of that structure. This is the sharp form of a slogan from Chapter 12, that *structure is compressibility*. Kolmogorov complexity is what “compressibility” means when you push it to the theoretical floor.

The definition also hands you a clean, and initially startling, definition of randomness. A string is **random**, in the Kolmogorov sense, when its complexity is close to its length: when it is *incompressible*, when there is no program shorter than the string that produces it. Randomness is not a property of a source spitting out coin flips; it is a property of the *individual object*, and it means precisely “no exploitable structure.” This is worth sitting with, because it inverts a common intuition. People expect a “complex” object to be ornate and a “random” one to be simple or degenerate. It is the reverse. A random string is *maximally* complex, because complexity here counts the length of its shortest description, and a random string’s shortest description is the string itself. Structure is what lets you say less; randomness is what forces you to say everything. Chapter 2 already whispered this: descriptions are countable and the reals are not, so almost every real number is incompressible, a countable dust of nameable numbers in an uncountable sea of numbers with no short description at all. Kolmogorov complexity is the tool that turns that whisper into a measurement.

Kolmogorov complexity vs. Shannon entropy

You have already met an information measure, Shannon entropy from Chapter 12, and it is essential not to blur the two, because they answer different questions about different kinds of thing.

Shannon entropy is a property of a *source*: a probability distribution. It measures the average description length for messages *drawn from that source*, the expected number of bits per symbol once you know the statistics. It presupposes a distribution and speaks about the ensemble.

Kolmogorov complexity is a property of an *individual object*: this specific string, full stop. It measures the description length of *that string*, with no reference to any distribution it might have come from.

Put crisply: Shannon tells you “on average, messages from this source need this many bits.” Kolmogorov tells you “this particular object needs this many bits, no matter what source, if any, produced it.” The two are deeply related (for a string generated by a simple probabilistic source, its Kolmogorov complexity is, on average, close to the source’s Shannon entropy) but they are not the same quantity, and the difference is exactly the difference between a statistical description and a mechanistic one, which is the difference this whole chapter exists to draw.

There is a price for Kolmogorov complexity’s generality, and it is the steepest price in the curriculum. Shannon entropy is computable: give me the distribution and I will compute H to as many digits as you like. Kolmogorov complexity is **uncomputable**: there is no general procedure that takes a string and returns K of it. This is not a temporary state of the art, not a hard problem

awaiting a clever algorithm. It is a proved impossibility, of the same family as the halting problem, and it is the reason the ultimate compression is a horizon you approach but never reach.

Why K is uncomputable, the argument

You now hold every piece needed to see why, and the proof is short enough to carry in your head. Suppose, for contradiction, that some program $\text{COMPLEX}(x)$ computes $K(x)$ exactly for every string x . Then build the following program, call it BERRY. It has a number n written into its own text, and it does this: search all strings in order of length, shortest first, and print the first string s for which $\text{COMPLEX}(s)$ is greater than n . Such a string always exists, because there are only finitely many programs shorter than n bits (fewer than 2^n of them) so only finitely many strings can have complexity below n , while strings themselves are unlimited; eventually you must hit one whose shortest program exceeds n bits, and BERRY prints it.

Now weigh BERRY's own length. It consists of a fixed chunk of code (the search loop and a copy of COMPLEX) plus the digits of the number n , so its total length is roughly a constant plus about $\log n$ bits to write n down. For n large enough, that total is *less* than n . But BERRY, being a program that prints the string s , is itself a description of s , so the shortest description of s can be no longer than BERRY. We have therefore exhibited a description of s shorter than n bits, while s was chosen precisely because *its shortest description exceeds n bits*. A string that provably has no description shorter than n has just been described in fewer than n bits. Contradiction. The only assumption we made was that COMPLEX exists, so it does not. Kolmogorov complexity cannot be computed by any program.

This is the old **Berry paradox** made mechanical: “the smallest positive integer not definable in under eleven words” is itself a definition, in ten words, of a number defined to need eleven. The paradox is defused in plain language by noting that “definable” is too slippery to be a real predicate, but Kolmogorov complexity *is* a perfectly precise predicate, so the same maneuver stops being a word game and becomes a proof. And notice the family resemblance one more time: like the halting saboteur, BERRY defeats the hypothetical decider by turning it on itself, feeding the machine a task designed around its own output. Same diagonal, fourth appearance. Cantor's list, Gödel's formula, Turing's D, and now BERRY: four objects, one recipe. Assume a complete list or an infallible decider; construct an object that systematically disagrees with it at a point it cannot escape; read off the contradiction. When you meet this pattern a fifth time in your own work, you will recognize it, and recognizing it is a large part of what a map is for.

Generative equations as programs

Now the payoff, and it is your own idea returning with a proof behind it. When you find the rule that generates a dataset, the equation governing the process, you have found a *short program* that produces the data. Suppose your data is a sequence of measurements y_1, y_2 , up to y_n , and you discover that each y_i equals $f(x_i)$ plus noise for some compact function f . Then the “program” that regenerates the systematic part of your data is just “compute f at each x_i ,” which is far shorter than the list of all the y_i . The data is the output; the equation is the program; and the length of that program is close to the Kolmogorov complexity of the systematic part of the data. Finding the equation is recovering the program from its output, which is, formally, *decompression*: you are handed the compressed file (the data) and you reconstruct the generator (the rule).

This is exactly why generative equations are more powerful than statistical summaries, and now you can say *why* in three registers at once. Structurally (Chapter 14), the equation is a stronger model because it could not have fit data from a different generating process, so its evidence is

enormous. Information-theoretically (Chapter 12), the equation achieves compression by finding the mechanism rather than the pattern, and mechanism compresses not just the sample but all future output of the same process, so its description length is minimal. Algorithmically (this chapter), the equation is a short program near the Kolmogorov floor, whereas a statistical fit is a longer description that has memorized surface regularities. The three registers are one claim: *capturing how the data was made beats capturing how the data looks*, because the maker is a short program and the look is a long one.

Worked example: logistic growth. Return to the bacterial growth curve from Chapter 14. A statistical description might store a smoothing spline with forty knots, or a lookup table of density at each timepoint; that is a long file, and it generalizes to nothing outside the sampled window. The generative description is $dN/dt = r \cdot N \cdot (1 - N/K)$, two parameters, r and K , and an initial condition. That is a handful of numbers and one line of structure, and it regenerates the entire curve at any resolution, forward past the last measurement, backward before the first, at timepoints you never sampled. The spline is a photograph of the data; the differential equation is the organism's growth logic. In Kolmogorov terms, the differential equation is a dramatically shorter program with the same output, so it is the better compression, and better compression here means genuine understanding, because the shortest program is the mechanism and nothing but the mechanism. This is precisely the discrimination the chapter's assignment asks you to perform on your own data: not "does it fit" but "is this the generator or merely a good photograph," and the deciding evidence is whether the candidate keeps predicting where you have no data, which is the operational signature of having recovered a program rather than a pattern.

One honest caution, owed to Chapter 10. A short generating program does *not* guarantee a long-range prediction. A chaotic system obeys an exact, compact rule and yet its trajectories separate exponentially, so "I know the equation" and "I can predict the future" are different claims: the rule is short, but running it forward accurately for a long time can demand impossibly precise initial data. Kolmogorov complexity measures the length of the shortest generator, not the cost of running it (that was the time-complexity shelf from the disambiguation box), and a system can sit at low descriptive complexity while being practically unpredictable. Recovering the mechanism is the deepest form of compression; it is not a promise of prophecy.

The search space of equations

You asked whether the search space of equations itself could be made variable, and that question is a live research frontier, not a settled technique. Here is the landscape it lives in.

The standard approach in **symbolic regression** (the discipline behind tools like Eureqa and PySR) is to search over a fixed *grammar* of mathematical expressions: a set of primitive operations (say $+$, $-$, $\times$, $\div$, $\sin$, $\cos$, $\exp$, $\log$, and powers) together with rules for combining them and the variables into expression trees. The grammar defines the search space; every candidate equation the search can ever propose is some legal tree built from those primitives. You can feel the whole engine here, and its central limitation with it: *you can only find equations expressible in the operations you put in the grammar*. The choice of grammar biases the search before it begins.

That single observation opens the frontier questions, and yours is squarely among them.

1. **Grammar design.** What operations belong in the grammar? If the true law involves a Bessel function and your grammar has only polynomials and trig, the search cannot find it, no matter how long it runs. Include everything and the space explodes. Every inclusion is a bet, and you are betting before you have seen the answer. In your own field you already prune this space with dimensional analysis: if a rate has units of one over time and your variables carry known units,

most syntactically legal expressions are dimensionally nonsense and can be struck out before the search ever scores them. Units are a piece of structural knowledge that shrinks the grammar honestly, and honoring them is the difference between a search that finishes and one that does not.

2. **Variable search space.** This is your question. Can the grammar itself *evolve*, so the search discovers new operations, not merely new combinations of a fixed set? This is the territory of *program synthesis* and *meta-learning*: instead of searching for an expression within a language, you search, at a higher level, for the language. Some recent work on neural program synthesis and on learning reusable primitives (building a library of useful sub-programs and then composing them) moves in exactly this direction. The reason your instinct was sound is that a fixed grammar caps the descriptive complexity the search can ever express, and a variable grammar is an attempt to lift that cap from inside the search itself.
3. **The gap between expressiveness and searchability.** Here is the fundamental trade, and it is where your question gets its teeth. A larger grammar can *express* more equations but is *harder to search*; a smaller grammar is easy to search but may not contain the truth. This is the bias-variance trade-off from Chapter 14 wearing new clothes, and it is best understood through the disambiguation box that opened this chapter. Expressiveness is about *descriptive* complexity: can the grammar even write the equation down? Searchability is about *time* complexity: can you find it before the heat death of the universe? Your frontier question lives precisely at the collision of these two axes, which is why it connects to the hardness side of Chapter 5 (P vs. NP, the question of whether easy-to-check means easy-to-find) and why it reappears as open question number 4 in Chapter 17. Making the search space variable does not abolish the trade; it moves the battle up one level, and the same wall (a space rich enough to hold the answer is too large to sweep) is waiting there too. This is your intuition about *powerful versus brute-force operations*, now made exact: a powerful move finds the mechanism along the grain of the problem, while brute force sweeps an exponential space; symbolic regression is the attempt to automate the search for the powerful move, and its difficulty is the price of not already knowing where the grain runs.
4. **Solomonoff induction.** The theoretical ideal at the far end of all of this. Assign to each program p a prior probability of 2 to the power of minus $|p|$, so shorter programs are more probable, exponentially so, and then update on the observed data by ordinary conditioning. The posterior provably converges on the true generating program in the limit, given enough data. This is Bayesian Occam's razor from Chapter 14 taken to its logical conclusion: "prefer the shorter description" becomes "weight every program by how short it is and let the data decide." It is the mathematically optimal way to learn a generator from observations. It is also, being built on program length, *uncomputable*, so every practical method (symbolic regression, minimum description length, the compression heuristics you already use) is an approximation of this uncomputable ideal, trading optimality for the ability to actually run.

What this means for your map:

Stand back and the whole landscape of "learning from data" resolves into a single ladder of ambition, each rung asking a stronger question than the one below, each harder to reach and more powerful once reached.

- **Statistics** asks: what distribution does this data come from? Its tools are Shannon entropy and estimation. It captures the source.

- **Machine learning** asks: what function maps inputs to outputs? Its concerns are approximation and generalization. It captures the input-output map.
- **Symbolic regression** asks: what equation generates this data? Its measure is Kolmogorov complexity, its goal compression. It captures the mechanism as an equation.
- **Solomonoff induction** asks: what program generates this data? Its instrument is the universal prior, its aim the ultimate compression. It captures the mechanism as a program, at the theoretical limit.

These are nested levels of ambition, and each subsumes the previous: a program is more general than an equation, an equation more general than an input-output function, a function more expressive than a bare distribution. Each rung is harder to climb and more powerful when reached, and the top rung is provably out of reach, which is not a defeat but an orientation. It tells you that the search for generative equations, the thing you intuited on your own, is not a niche technique but the computable shadow of the deepest possible way to learn, and that its difficulty is principled, written into the mathematics, not a failure of your tools.

Where you now stand

You entered this chapter with a self-taught intuition (that discovering a generative equation is a uniquely powerful form of compression) and a frontier question (whether the search space of equations could itself be made variable). You leave with both placed exactly on the map and equipped with proofs. You have a precise, machine-independent measure of the information in an object, Kolmogorov complexity, the length of the shortest program that outputs it, and with it a clean definition of randomness as incompressibility and a sharp line between Shannon's statistics of a source and the algorithmic content of an individual object. You have seen why the ultimate compression is uncomputable, and you have seen it is the *same* impossibility as the halting problem, both of them the *same* diagonal that gave you uncountable infinities and unprovable truths: assume the decider, build the object that defeats it by self-reference, collect the contradiction. You can now say, in three reinforcing registers, why capturing the mechanism beats capturing the pattern, and you can locate your own frontier question at the collision of expressiveness and searchability, which is the collision of descriptive complexity with time complexity, which is the door to P vs. NP and to the open questions of Chapter 17.

One structural observation has now recurred in every phase of this curriculum and been named in none: the *right maps between structures are the ones that preserve the structure*. Measurement was a homomorphism, a model was a homomorphism, compression is finding the map that preserves everything while shrinking the description, and the halting and complexity results are all facts about maps between the space of programs and the space of their outputs. Chapter 16, category theory, takes that single recurring idea and makes it the primary object of study, the mathematics of the structure-preserving map itself, the language in which every connection you have drawn across these fifteen chapters can finally be stated as one thing.